TerrainForge — Performance & Reproducibility Benchmark Study

A quantitative evaluation framework for a browser-based procedural terrain generation engine

Project Title: CPU-Bound Procedural Terrain Synthesis with Real-Time Algorithmic Deformation [TerrainForge]

---

1. Abstract

Generating realistic and interactive 3D terrain with rivers, water systems, erosion, and dynamic environmental events commonly depends on powerful GPUs, substantial computational resources, and specialized or paid software. This creates an accessibility barrier for students, schools, and researchers working with older or resource-constrained devices. TerrainForge is a browser-based, CPU-driven procedural terrain-generation and simulation system designed to investigate whether mathematically structured algorithms can provide visually complex and interactive terrain without requiring dedicated graphics hardware.

The TerrainForge generation pipeline begins with a deterministic numeric seed and combines multiple procedural height-field techniques, including fractal Brownian motion (fBm), Ridge, Billow, Voronoi-based structures, and custom mathematical equations. The resulting height field can then undergo multiple levels of erosion and hydrological processing. Laplacian smoothing provides a lightweight method for suppressing high-frequency terrain noise; thermal/talus erosion redistributes material when local slopes exceed a specified stability threshold; and hydraulic droplet erosion models water-driven sediment transport through virtual droplets that follow interpolated terrain gradients, eroding, transporting, and depositing sediment along their paths. Flow accumulation and river-flow carving further transform the height field into recognizable drainage structures. A biome-classification and splat-mapping stage then assigns environmental characteristics and visual materials before the final terrain is converted into a 3D mesh.

A significant computational component of TerrainForge is its node-graph compiler, which allows users to construct procedural terrain equations through a visual node editor. Rather than simply concatenating nodes in graph order, the compiler performs a three-pass optimization pipeline. First, Kahn's topological sorting algorithm establishes dependency order and detects cycles in the user-defined graph. Second, backward reachability analysis from the Output node performs dead-code elimination, removing nodes that cannot influence the final terrain expression. Third, expression generation incorporates common-subexpression elimination (CSE) and strength reduction, allowing structurally identical expressions to be reused and mathematically redundant operations such as addition by zero or multiplication by one to be removed. The resulting optimized expression is subsequently compiled into the terrain-generation evaluator.

TerrainForge additionally introduces a real-time Chaos Engine for dynamic terrain response to events such as impacts and tsunami-like disturbances. The engine evaluates a coupled partial differential equation (PDE):

$$\partial T/\partial t = \alpha \cdot E_w \cdot \nabla^2 T - \beta \cdot |F_{dir}| \cdot (1 - \text{stability} \cdot 0.6) - \delta \cdot \nabla^4 T$$

The model combines curvature-driven diffusion weighted by wave energy, a directional sediment-transport component influenced by slope stability, and a fourth-order smoothing component intended to suppress isolated numerical noise while preserving larger-scale terrain structure.

The system was evaluated through multiple performance and reproducibility experiments on a Samsung Galaxy Tab A7 Lite and a Vivo 1820, representing older, resource-constrained Android hardware. Across four terrain-resolution tiers, observed rendering performance ranged from the high teens to the low 30s FPS on both devices. A cross-device determinism experiment using the same seed produced identical vertex count (6,561), triangle count (12,800), peak height (4.95 units), water coverage (53% at level 0.80), and biome classification (Snowcap) at a fixed test coordinate on both devices — an aggregate-and-fixed-point reproducibility check, not a complete per-vertex numerical comparison. Enabling the real-time Chaos Engine changed frame rate by +0.56fps (Samsung, 31.00→31.56) and +0.33fps (Vivo, 25.67→26.00), both small increases within ordinary trial-to-trial noise rather than the measurable CPU-scaled cost the study set out to test for.

Overall, TerrainForge investigates a low-resource approach to procedural terrain synthesis by combining deterministic procedural generation, physically inspired erosion models, hydrological processing, visual node-graph compilation, mathematical expression optimization, and real-time PDE-based environmental simulation. While the reproducibility experiment and hardware evaluation are limited to the two tested configurations, the findings provide evidence that procedural terrain systems do not necessarily require high-end GPUs to produce interactive, visually detailed results.

PROJECT WEBSITE & MATERIALS
https://terrainforge-research.github.io/TerrainForge-research-project/

---

2. Introduction

2.1 Background

Procedural generation allows software to create large, varied 3D content from compact mathematical rules — noise functions, erosion simulations — instead of hand-authoring every asset. This approach is used in games, GIS/education tools, and simulation software. Because tools like this increasingly run inside a web browser rather than a native game engine, and are used on hardware ranging from budget phones to gaming desktops, their real-world usability depends heavily on measured performance, not just on the visual quality of a single demo screenshot. This motivates a controlled, repeatable benchmark rather than an informal impression of "it feels fast."

2.2 Application Under Test: TerrainForge Pro

The following description is drawn directly from inspection of the application’s source file.

Aspect	Detail
Delivery	Single self-contained HTML/CSS/JS file (~11,300 lines); no installation, no backend server
Rendering	WebGL 3D scene via Three.js r128 (CDN), real-time shadow mapping, exponential fog, ACES Filmic tone mapping
Generation pipeline	Seed -> Base Height (fBm/Ridge/Billow/Voronoi or custom equation) -> Erosion -> Rivers -> Biome Splat -> Mesh
Determinism	Every world is seed-driven; the application's own UI claims a shareable "map code" reproduces an identical world, vertex-for-vertex, on any device
Export formats	OBJ; GLB at four mesh tiers; 16-bit heightmap; biome texture; per-vertex data-channels GLB; RGBA splatmap
Interaction	Orbit camera, click-to-inspect, first-person walk mode, live Analysis tab

Aspect	Detail
Real-time simulation	A "Chaos Engine" evaluates a user-editable PDE over the heightmap every frame; the app's own roadmap notes this is currently CPU-bound
Built-in instrumentation	A Generation Benchmark panel reports Grid Resolution, Vertex Count, Triangle Count, and Generation Time — used directly as a measurement instrument (§6.2)

Note: the "vertex-for-vertex" phrase above is the application's own UI claim, reported for context — this study's own determinism test (§7.6) measures five aggregate/point values, not a full per-vertex comparison; see §11.

2.3 Problem Statement

Realistic terrain generation is usually associated with powerful computers, dedicated GPUs, and specialized software. This becomes a practical problem when the goal is to experiment with terrain generation on older or low-resource devices. A student may be able to create a simple heightmap, but generating terrain that also has believable rivers, erosion, water flow, different biomes, and dynamic environmental effects becomes much more computationally demanding. The main problem addressed in this project is therefore: can realistic, interactive 3D terrain with erosion, water flow, river formation, biome classification, and dynamic environmental effects be generated and simulated on ordinary, low-resource hardware using CPU-based computational methods?

There is also a second problem within the system itself. Procedural terrain is not just a matter of generating random noise. Different mathematical operations, filters, erosion models, and transformations have to work together. When these operations are connected through a visual node system, unnecessary calculations can easily accumulate. A poorly ordered or redundant graph can increase computation without changing the final terrain. TerrainForge approaches this by combining procedural height-field generation with thermal and hydraulic erosion, flow accumulation, deterministic seeding, and a visual node-graph compiler using topological sorting, dead-code elimination, common-subexpression elimination, and strength reduction.

2.4 Research Question

To what extent can a browser-based procedural terrain pipeline — including height synthesis, erosion, real-time terrain deformation, and a user-facing equation compiler — run on a single CPU thread while maintaining interactive performance on low-end Android hardware, and under what conditions does its performance begin to break down?

2.5 Objectives

1. Measure rendering performance by recording FPS across four controlled terrain-resolution tiers on the available low-end Android devices.

2. Measure terrain-generation time and determine how generation latency changes with terrain resolution and different erosion configurations.

3. Characterize erosion scaling by measuring the computational cost of increasing hydraulic-erosion droplet counts and thermal-erosion iteration counts.

4. Evaluate export performance by measuring export time and resulting file size across the supported OBJ, GLB, heightmap, biome-texture, data-channel, and splatmap formats/tiers.

5. Test deterministic reproducibility by generating the same seeded terrain on different devices and comparing key outputs including vertex count, triangle count, peak height, water coverage, and biome classification.

6. Measure the performance impact of the real-time Chaos Engine by comparing baseline FPS with FPS during Chaos Engine operation under the same terrain configuration.

7. Evaluate the Auto-LOD system by examining how rendering performance changes with camera distance and level-of-detail reduction.

8. Assess resource requirements by measuring memory footprint and initial load/time-to-interactive on the tested hardware.

9. Identify performance limits and breakdown conditions by determining which combinations of terrain resolution, erosion workload, and real-time simulation produce unacceptable generation latency or loss of interactive frame rate.

10. Evaluate the feasibility of CPU-only procedural terrain generation on low-resource hardware by relating visual complexity, computational workload, reproducibility, and measured performance.

2.6 Novelty / Innovation

The engineering contribution under test is not a claim of physical accuracy; it is that four specific techniques let a real, discretized simulation run on hardware that could not run a naive version of it:

A compiled node-graphfor user-authored terrain equations — dependency resolution via Kahn's topological sort, dead-code elimination via backward reachability from the Output node, and common-subexpression elimination via fingerprint memoization, all folded into one expression-generation pass rather than run as separate string-processing steps.

A zero-allocation real-time deformation loop — every scratch buffer used by the "Chaos Engine" tick is a pre-allocated Float32Array reused every frame (.fill(0), not re-allocated), specifically to avoid garbage-collector pauses during continuous simulation.

A layered, explicitly-labelled stability design for the deformation PDE — curvature and hyper-diffusion terms are each bounded by a tanh saturation before being scaled, and the combined per-tick height change is clamped a final time, rather than relying on a single safeguard or a costly implicit solver.

Deliberate procedural approximation of environmental events (tsunami, cyclone, wave surface) using closed-form parametric functions instead of solving shallow-water or Navier–Stokes equations, an explicit accuracy-for-speed trade-off rather than an unstated shortcut.

2.7 System Architecture / Pipeline

Generation proceeds through five stages, entirely client-side, in a single HTML file with no build step or server dependency: a seed drives noise-based or node-graph-compiled height synthesis; particle-based hydraulic erosion and a separate thermal pass shape the heightfield; the result is meshed at a user-selected resolution tier; biome colouring is applied by height and slope; and an optional real-time layer allows the mesh to deform live in response to injected disturbances. The two figures below are direct output from this pipeline, captured on the Samsung Galaxy Tab A7 Lite test device (§6.3), not renders from a separate demo build.

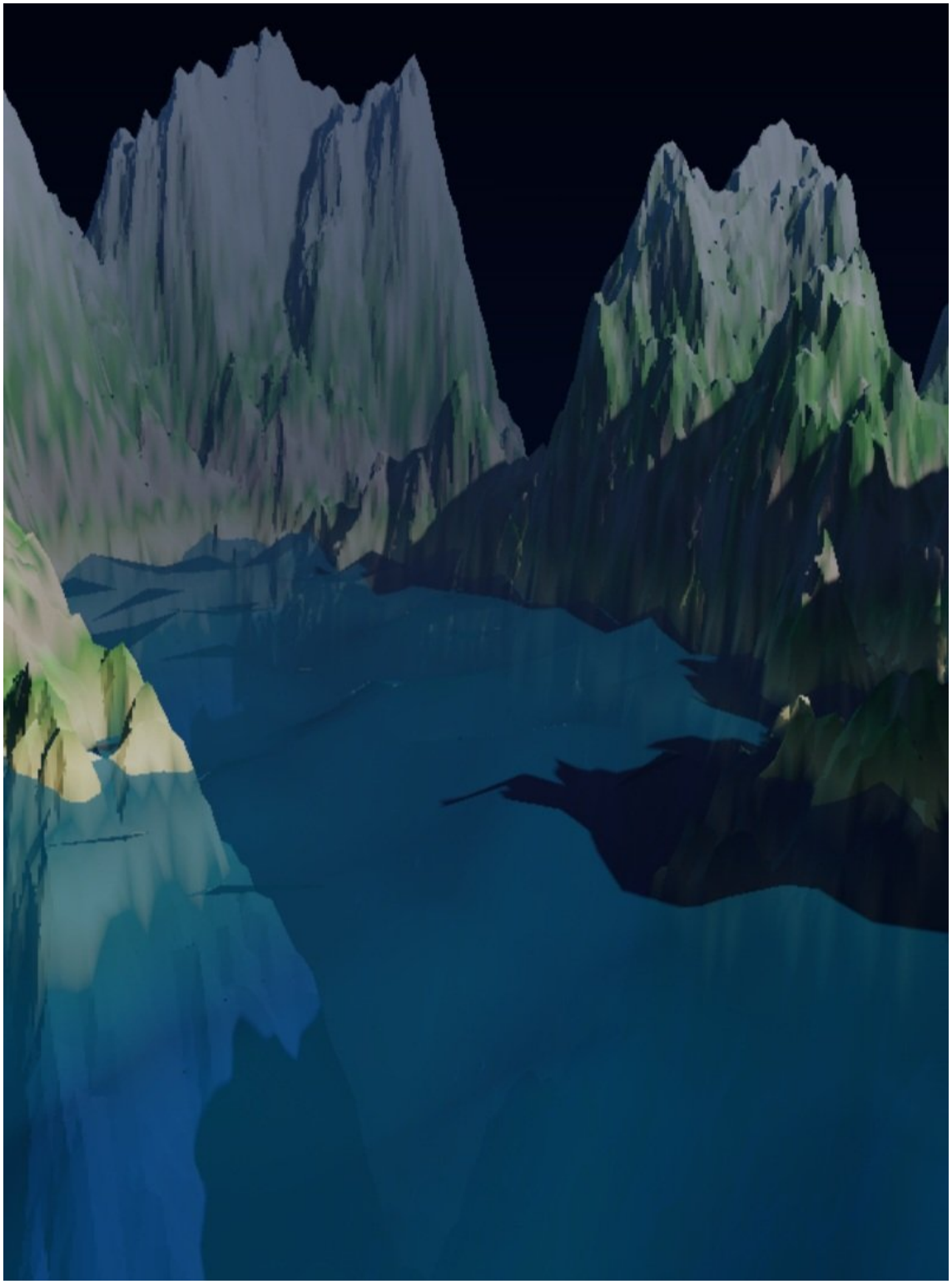


Figure 1. Procedurally generated terrain and water-body visualization using TerrainForge on a Samsung Galaxy Tab A7 Lite at 224×224 resolution. The visualization incorporates custom terrain equations for water formation, thermal erosion, biome classification, and custom colour mapping, demonstrating the system's integrated terrain-generation, erosion, hydrological, and biome-rendering pipeline.

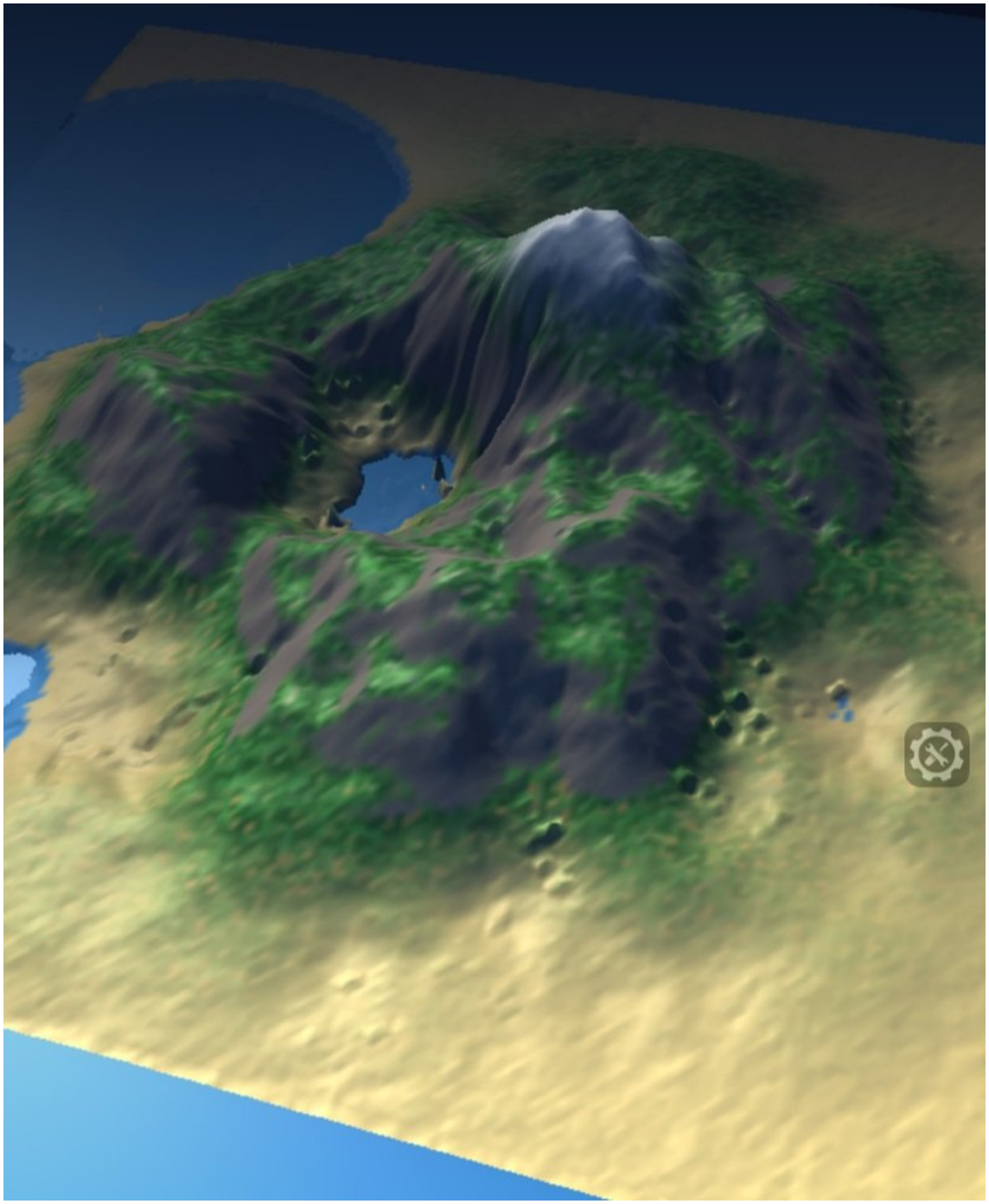


Figure 2. Erosion-model output generated by TerrainForge, showing pronounced slope modification, incised terrain features, and low-lying water accumulation within the generated landscape. The visualization demonstrates the morphological effects produced by the implemented erosion pipeline and its interaction with terrain elevation and water-body formation.

2.8 Key Technical Methodology

This subsection describes how the system itself works; §7 (Methodology) describes how it was measured, which is a separate concern.

Node-graph compiler. Kahn’s algorithm (in-degree tracking, queue-based removal) orders node evaluation and detects cycles as a side effect of the same bookkeeping; a backward depth-first search from the Output node prunes unreachable nodes before code generation; a fingerprint of each node’s type, parameters, and already-compiled input expressions is checked against every fingerprint seen earlier in the same compile, and a match reuses the existing expression instead of duplicating it.

Hydraulic erosion. A particle/droplet method (Beyer, 2015): a droplet carries inertia, a sediment capacity proportional to its speed and remaining water, and evaporates geometrically each of up to 32 steps; it deposits when over capacity or moving uphill, and erodes otherwise.

Real-time deformation ("Chaos Engine"). A discretized surface-update rule, evaluated every nth render frame rather than every frame:

$$\partial T/\partial t = \alpha \cdot E_w \cdot \nabla^2 T - \beta \cdot |F_{dir}| \cdot (1 - \text{stability} \cdot 0.6) - \delta \cdot \nabla^4 T$$

The curvature term ($\nabla^2 T$, a 5-point Laplacian) and the fourth-order term ($\nabla^4 T$, a 13-point biharmonic stencil, weights 20/−8/+2/+1, which sums to exactly zero) are each passed through a tanh bound before being scaled; the combined per-cell height change is clamped a final time before being applied. This layered bounding, not raw compute, is what §7.7 (Test F) measures the cost of.

2.9 The Chaos Engine — A Closer Look

Every other part of TerrainForge runs once: set some sliders, click Generate, and the terrain is fixed from then on. The Chaos Engine is the one part that doesn't stop there. Once it's switched on, it keeps re-running the deformation equation over the terrain, tick after tick, so the ground keeps changing while it's being watched — peaks wear down, material piles up somewhere else, and the surface settles or doesn't depending on how it's tuned. It's the difference between a snapshot of a landscape and a landscape that keeps responding.



Figure 3. The Chaos Engine control panel, captured mid-run: the governing equation, the coupling-parameter sliders, and the live stats (Peak Energy, Stability, Volume Eroded, Volume Deposited) after 11 coupling steps.

Reading the control panel. The equation shown on the panel itself is:

$$\partial T/\partial t = \alpha \cdot E_w \cdot \nabla^2 T - \beta \cdot |\nabla W| \cdot (1 - \text{stability}) + \gamma \cdot \text{sediment} - \delta \cdot \nabla^4 T$$

Every term on that line has its own slider in the "Coupling Parameters" section, using the same Greek letter:

α — Erosion (1.40). How hard curvature gets acted on. Turn this up and sharp peaks and pits get smoothed toward their surroundings faster.

β — Lateral (0.80). How much the wave pushes material sideways rather than straight up or down. Higher values drag more sediment across a slope instead of only along it.

γ — Sediment (0.40). The term that closes the loop between erosion and deposition. Material the curvature term picks up doesn’t just vanish — this is what actually adds it back in somewhere else, rather than only ever removing it.

δ — Hyper-Diff (0.60). The cleanup term — the panel’s own subtitle calls it what "kills blocky noise." It knocks down single-cell spikes and pits without flattening real hills, because it only reacts to very sharp, small-scale bumps, not broad slopes.

Wave Scale (0.60) and Wave Speed (1.20). How far the disturbance reaches across the terrain, and how fast it travels once it starts.

Energy Cap (0.028). A hard ceiling on how much any single tick is allowed to change the terrain by, regardless of how energetic the wave gets — the safeguard that keeps the simulation from ever running away into an unusable result.

Tick Rate (10). How many of these updates run per second while the engine is active.

Below the sliders, four numbers report what actually happened, rather than what’s configured. Peak Energy is the highest wave-energy value reached during the run. Stability reflects how well-behaved the simulation stayed rather than drifting toward runaway change — 98% here means it stayed close to fully settled. Volume Eroded and Volume Deposited report how much terrain was picked up versus put back down, in the same units as the heightmap itself. In this run the two are close but not equal — about 205,042 eroded against 197,845 deposited — so slightly more material left the system than the γ term returned to it.

Low energy versus high energy. Wave Energy (E_w) sits directly in front of the curvature term, multiplying it. That’s the entire effect of that one value: it doesn’t change what the equation does, only how hard it does it.

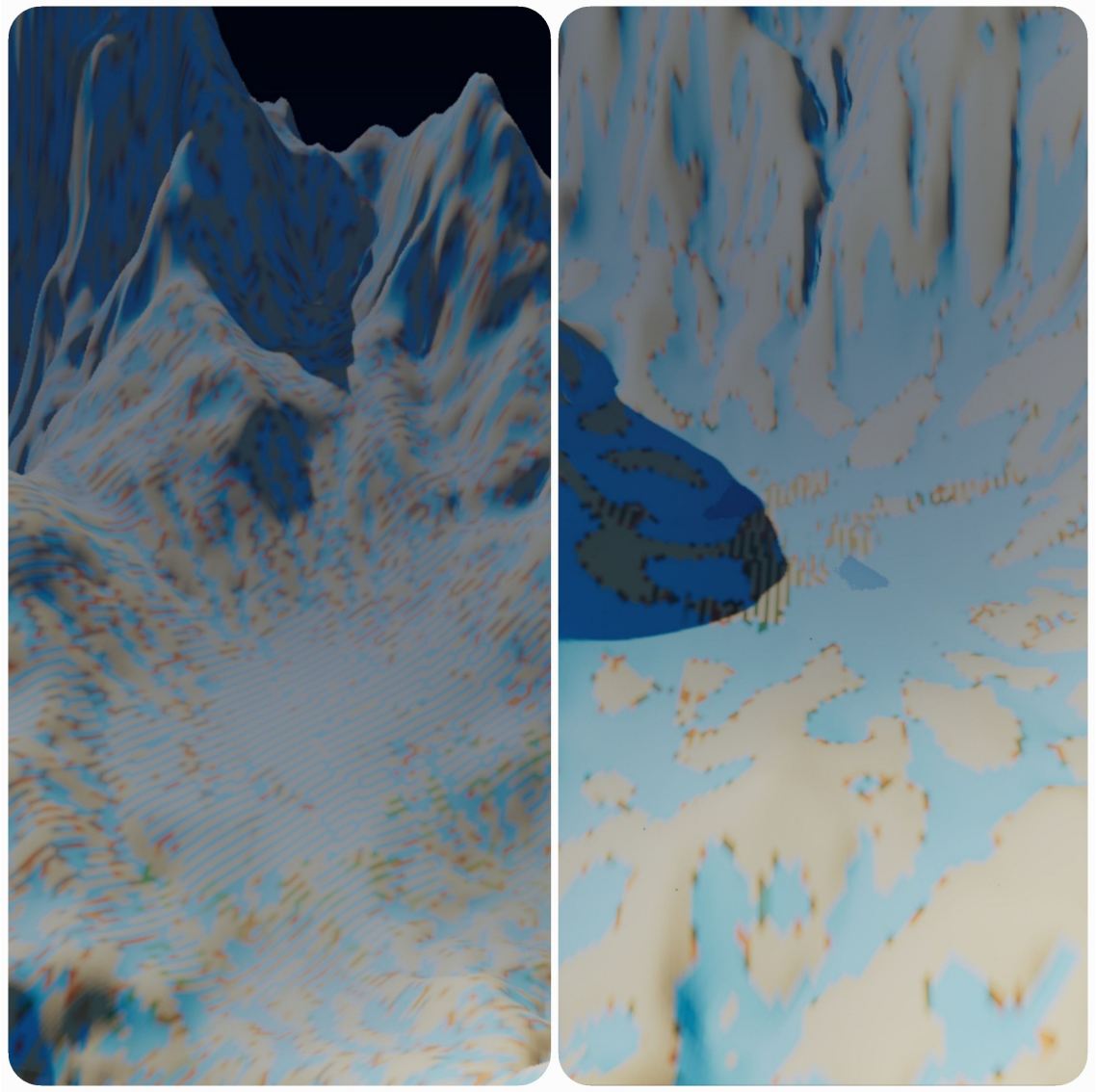


Figure 4. The same curvature-driven deformation at low wave energy (left) versus high wave energy (right). Same equation, same terrain, only the E_w multiplier changed.

At low energy, the terrain keeps its ridgelines and defined peaks largely intact — the curvature term is only lightly weighted, so it nudges the surface rather than reshaping it. At high energy, the same term is weighted so much harder that peaks and pits get pulled toward the local average faster than real structure can be told apart from noise, and the terrain reads as washed-out rather than sculpted. That’s the actual trade-off the slider controls: low energy behaves like gradual weathering; high energy pushes the same mechanism hard enough that it stops looking like erosion and starts looking like the terrain melting toward flatness.

Before and after. The two images below show one terrain immediately before and after a Chaos Engine run, from the same vantage point.

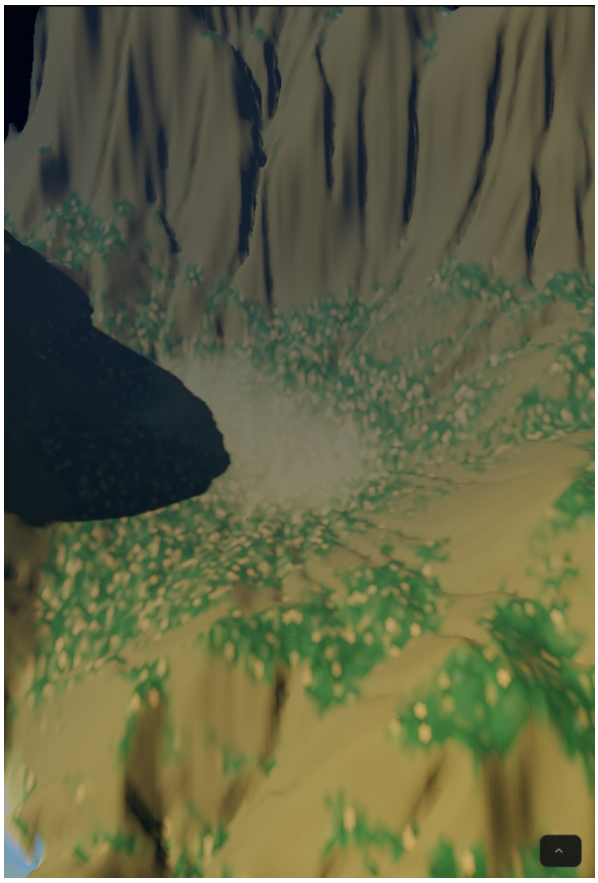


Figure 5. Before the Chaos Engine coupling was enabled.

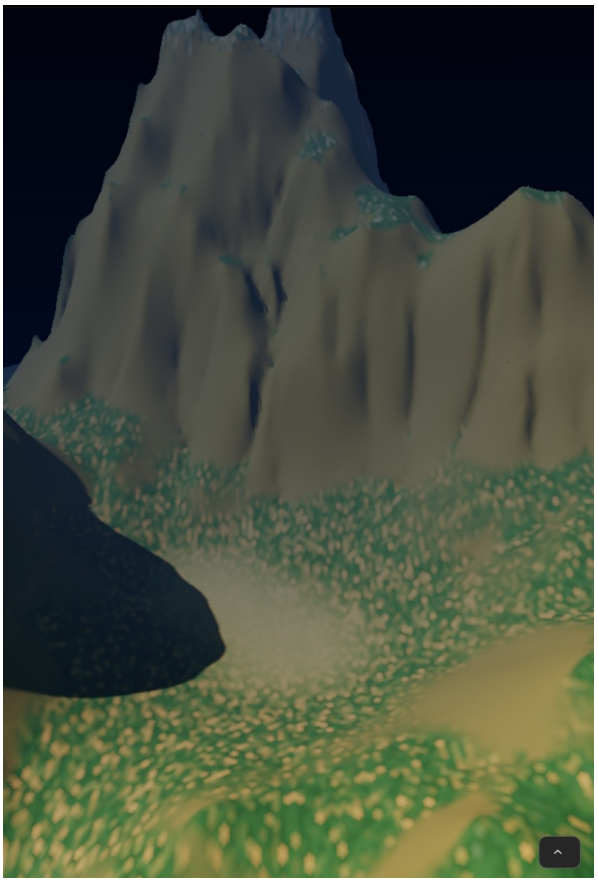


Figure 6. After the run shown in Figure 3 — 11 coupling steps, Seismic wave mode, Wave Energy 3.77.

---

3. Hypotheses

H1: Rendering frame rate (FPS) will decrease as triangle count increases, and the decline will be steeper on devices with weaker/integrated GPUs than on devices with discrete GPUs.

H2: Terrain generation time will increase with grid resolution, and will increase further when hydraulic and/or thermal erosion is enabled, since these run as additional passes over the heightmap.

H3: Hydraulic erosion time will scale approximately linearly with droplet count; thermal erosion time will scale approximately linearly with iteration count, at a fixed resolution.

H4: An identical seed/map code will reproduce identical vertex counts, triangle counts, and terrain statistics (e.g., peak height) across all tested devices, supporting the application's determinism claim.

H5: Activating the real-time Chaos Engine simulation will measurably reduce FPS relative to a static scene at the same resolution, with a proportionally larger reduction on lower-end CPUs.

H0 (Null, applies to each test above): There is no statistically significant relationship between the tested independent variable and the measured output.

These are starting hypotheses grounded in the application’s architecture, not guaranteed outcomes — §10 (Discussion) reports which were supported by the data actually collected.

---

4. Variables

4.1 Independent Variables

Variable	Levels / Range Tested	Source in Application
Terrain grid resolution	32 / 80 (default) / 160 / 224 (max)	"Resolution" slider, Terrain tab
Erosion type	None / Thermal / Hydraulic / Both	Erosion tab toggle
Hydraulic droplet count	500 / 3,000 (default) / 8,000 / 15,000	"Droplets" slider
Thermal iteration count	5 / 20 (default) / 40	"Iterations" slider
Export format / tier	OBJ, GLB (4 tiers), Heightmap, Biome Texture, Data-Channels GLB, Splatmap	Export menu
Device / hardware tier	Defined by researcher — see §6.3	—
Auto-LOD state	On / Off	"Auto LOD" toggle, Terrain tab
Camera distance	Near / Mid / Far	Orbit camera

4.2 Dependent Variables (Outputs)

Output	Unit
Rendering frame rate	FPS
Terrain generation time	ms
Erosion-attributable time	ms (derived)
Export time	ms
Exported file size	KB / MB
Reproducibility match	Boolean (Y/N)
Simulation FPS overhead	FPS, %
Auto-LOD FPS gain	FPS, %
JS heap memory delta	MB
Time-to-interactive	s

4.3 Controlled Variables

Variable	How It Is Held Constant
Application build/version	Same, unmodified TerrainForge-Pro.html file for every trial
Three.js version	r128 (fixed CDN dependency inside the file)
Device pixel ratio	Capped at 1x by the application itself
Shadow map resolution	Fixed at 2048x2048 (hard-coded)
Power source	Devices on mains power for every trial
Background applications	Closed/minimized before each trial
Seed	Fixed test seed for every trial, unless seed is itself the variable
Map equation / noise type	Fixed at default (fBm) unless otherwise stated
Trials per condition	3 minimum

---

5. Measurement Framework (Outputs & Methods)

This table is the central reference for the whole study: what is measured, in what unit, using what instrument, and which test (§7) produces it.

#	Output	Unit	Measurement Method	Test
1	Rendering Frame Rate	FPS	In-app FPS readout + DevTools Rendering stats	A
2	Terrain Generation Time	ms	Documentation tab -> Generation Benchmark	B
3	Erosion-Attributable Time	ms (derived)	Gen. Time (erosion ON) minus Gen. Time (erosion=None)	C

#	Output	Unit	Measurement Method	Test
4	Export Time	ms	Manual stopwatch / performance.now() console timer	D
5	Exported File Size	KB/MB	OS file properties	D
6	Reproducibility Match	Y/N	Manual comparison, Documentation + Coordinate panels	E
7	Simulation FPS Overhead	FPS, %	In-app FPS readout, before/after activation	F
8	Auto-LOD FPS Gain	FPS, %	In-app FPS readout at fixed camera distances	G
9	JS Heap Memory Delta	MB	DevTools Memory panel / performance.memory	H
10	Time-to-Interactive	s	DevTools Performance panel / stopwatch	I

---

6. Materials and Test Environment

6.1 Software Under Test

Item	Detail
Application	TerrainForge Pro -- Procedural Map Builder
File	TerrainForge-Pro.html (single-file, client-side)
Core rendering library	Three.js r128 (CDN: cdnjs.cloudflare.com)
Execution model	100% client-side; no backend/server component
Version / build tested	Version V1
Network condition during tests	Normal 4G internet network connection was used in both devices while testing.

6.2 Measurement Instruments

In-app FPS readout — Coordinate/Info panel (FPS row); updates roughly twice per second while the panel is open.

In-app Generation Benchmark panel — Documentation tab; reports Grid Resolution, Vertex Count, Triangle Count, and Generation Time for the most recent build.

Browser DevTools -> Rendering panel ("Frame Rendering Stats") — independent on-screen FPS meter used to cross-validate the in-app counter.

Browser DevTools -> Performance panel — for recording generation/export timelines.

Browser DevTools -> Memory panel / performance.memory — for JS heap size before/after generation.

Stopwatch / performance.now() console timer — for export time.

OS file properties — for exported file size.

6.3 Device Test Bench

Device ID	Category	CPU	GPU	RAM	OS	Browser	Display	Refresh
Samsung Galaxy Tab A7 Lite	Tablet	MediaTek Helio P35, 8 cores	PowerVR Rogue GE8320	2762 MB	Android 12	Chrome 99.0.4844.88	800x1340	60Hz
Vivo 1820	Smartphone	MediaTek Helio P22, 2.00GHz, 8 cores	PowerVR Rogue GE8320	1799 MB	Android 8.1.0	Chrome 101.0.4951.61	720x1520	60Hz

6.4 Standard Test Configurations

Resolution tiers — from the Terrain tab "Resolution" slider (range 32–226, step 16, default 80):

Tier	Slider Value	Grid Size	Vertices	Triangles
R1 - Minimum	32	33x33	1,089	2,048
R2 - Default	80	81x81	6,561	12,800
R3 - High	160	161x161	25,921	51,200
R4 - Maximum	224	225x225	50,625	100,352

Erosion presets — fixed parameter combinations used across Tests B and C:

Preset	Erosion Type	Droplets	Thermal Iterations
E0 - None	None	--	--
E1 - Thermal (min)	Thermal	--	5
E2 - Thermal (default)	Thermal	--	20
E3 - Thermal (max)	Thermal	--	40
E4 - Hydraulic (min)	Hydraulic	500	--
E5 - Hydraulic (default)	Hydraulic	3,000	--
E6 - Hydraulic (high)	Hydraulic	8,000	--
E7 - Hydraulic (max)	Hydraulic	15,000	--
E8 - Both (default settings)	Both	3,000	20

Export formats/tiers under test:

Code	Export Type	Target Triangle Count
X1	OBJ (current live mesh)	= active resolution tier
X2	GLB - Low	10,000
X3	GLB - Medium	50,000
X4	GLB - High	100,000
X5	GLB - Ultra	400,000
X6	16-bit Heightmap (PNG)	--
X7	Biome Texture (PNG)	--
X8	Data-Channels GLB	= active resolution tier
X9	Splatmap (RGBA weight PNG)	--

---

7. Methodology

7.1 General Testing Protocol

Load TerrainForge-Pro.html fresh (hard refresh / cleared cache) before the first trial on each device, unless load-time caching is itself the variable under test (§7.10).

Allow the initial scene at least 5 seconds to settle before recording any measurement (JIT/GPU-driver warm-up).

Set the fixed test seed and all controlled variables (§4.3) before changing the independent variable under test.

Run a minimum of 3 trials per condition. Discard and re-run any trial interrupted by an OS notification, browser hitch, or stutter unrelated to the app itself.

Record raw trial values first; compute Mean and Standard Deviation only after all trials for a condition are collected.

Allow a cooldown period between resolution/device switches to avoid thermal throttling contaminating later trials.

Keep the browser window focused and the tab active during measurement.

7.2 Test A — Rendering Frame Rate vs. Terrain Resolution

Objective. Measure steady-state FPS at each resolution tier (§6.4) on each device.

Device ID	Tier	Triangles	T1	T2	T3	Mean FPS	Std Dev
Samsung Galaxy Tab A7 Lite	R1	2,048	32	31	29	30.67	1.53
Samsung Galaxy Tab A7 Lite	R2	12,800	26	33	34	31.00	4.36
Samsung Galaxy Tab A7 Lite	R3	51,200	28	32	33	31.00	2.65
Samsung Galaxy Tab A7 Lite	R4	100,352	28	37	34	33.00	4.58
Vivo 1820	R1	2,048	22	24	23	23.00	1.00
Vivo 1820	R2	12,800	26	24	27	25.67	1.53
Vivo 1820	R3	51,200	20	18	17	18.33	1.53
Vivo 1820	R4	100,352	17	20	17	18.00	1.73

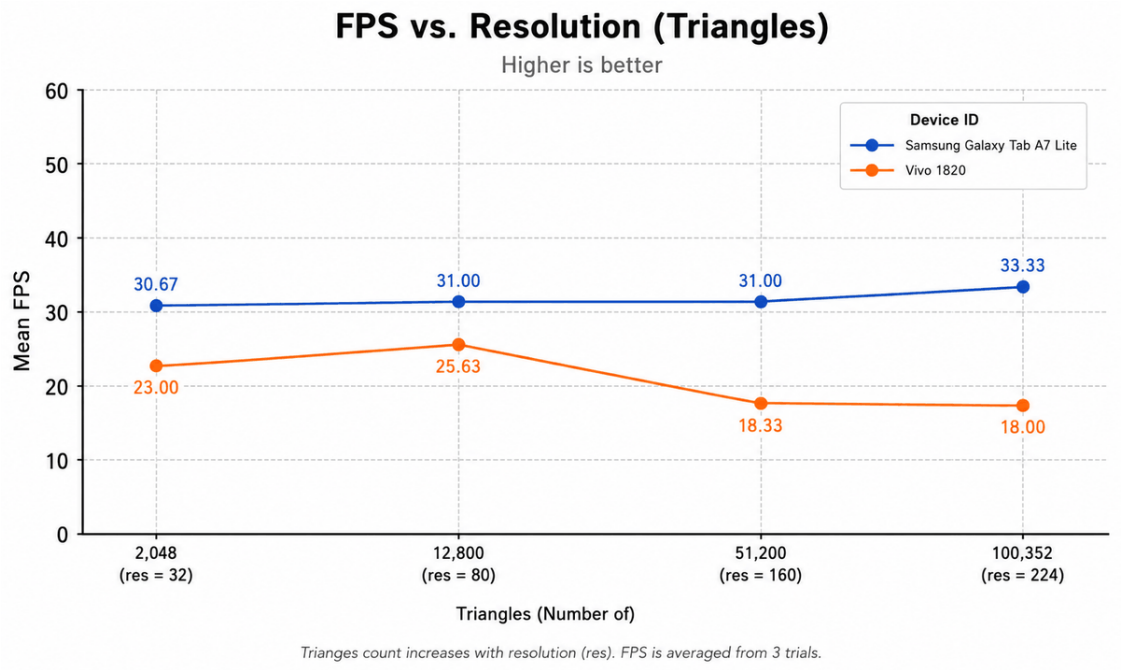


Chart. FPS vs. resolution, both devices.

7.3 Test B — Terrain Generation Latency

Objective. Measure end-to-end generation time via the built-in Generation Benchmark readout, isolating the effect of resolution and erosion type.

Device	Tier	Erosion	T1	T2	T3	Mean (ms)	SD
Samsung	R1	E0 None	1270	967	852	1029.67	215.93
Samsung	R2	E0 None	1020	1240	1720	1326.67	357.96
Samsung	R3	E0 None	1380	1110	1590	1360.00	240.62
Samsung	R4	E0 None	1560	1810	1370	1580.00	220.68
Samsung	R2	E2 Thermal	946	952	1390	1096.00	254.63
Samsung	R2	E5 Hydraulic	1160	1120	1270	1183.33	77.67
Samsung	R2	E8 Both	2720	3010	2330	2686.67	341.22
Vivo	R1	E0 None	1310	982	1230	1174.00	171.02

Device	Tier	Erosion	T1	T2	T3	Mean (ms)	SD
Vivo	R2	E0 None	1460	1220	1390	1356.67	123.42
Vivo	R3	E0 None	1330	1410	972	1237.33	233.24
Vivo	R4	E0 None	1890	1990	1910	1930.00	52.92
Vivo	R2	E2 Thermal	2260	2180	1920	2120.00	177.76
Vivo	R2	E5 Hydraulic	2540	2310	2490	2446.67	120.97
Vivo	R2	E8 Both	2850	2930	4250	3343.33	786.21

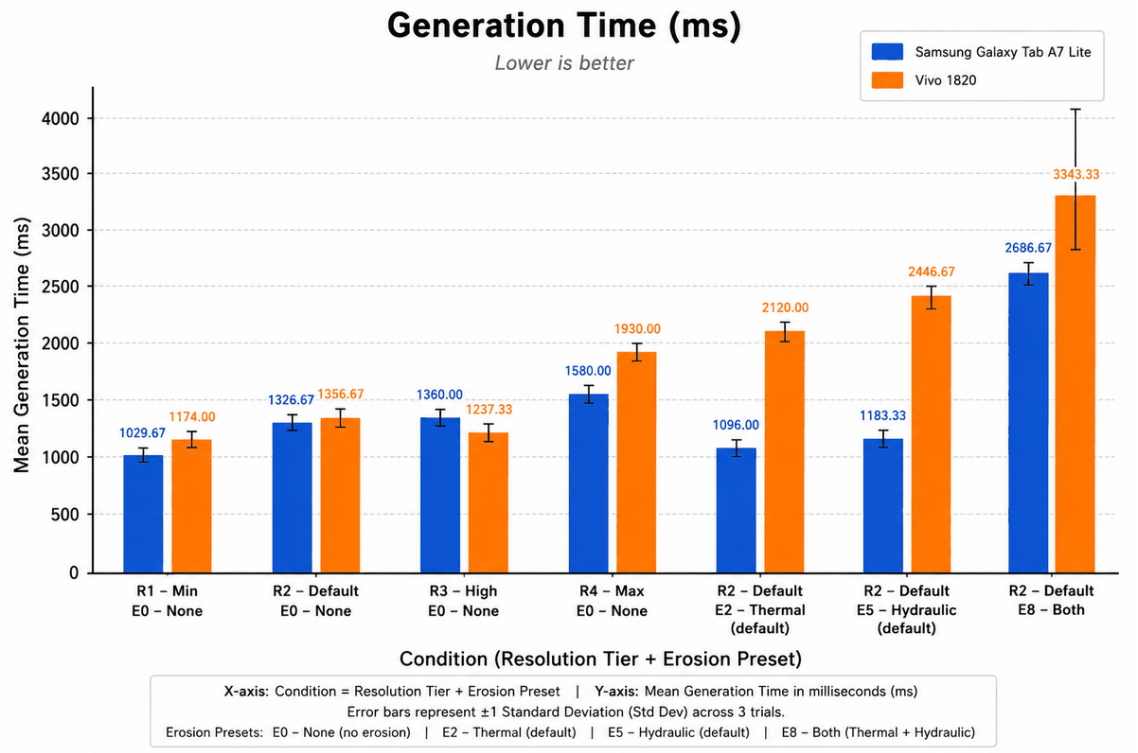


Chart. Generation time by condition, both devices.

7.4 Test C — Erosion Algorithm Computational Scaling

Objective. Quantify how erosion computation time scales with its own intensity parameter, independent of resolution. Resolution fixed at R2 for every row; Erosion-Attributable Time = Mean Generation Time (this row) minus Mean Generation Time at R2/E0-None for the same device (§7.3).

Samsung Galaxy Tab A7 Lite — Hydraulic

Droplets	T1	T2	T3	Mean (ms)	SD	Erosion-Attrib. (ms)
500 (E4)	647	661	732	680.00	45.57	-646.67
3,000 (E5)	700	1460	886	1015.33	396.16	-311.34
8,000 (E6)	766	838	860	821.33	49.17	-505.34
15,000 (E7)	1020	1490	1440	1316.67	258.13	-10.00

Samsung — Thermal

Iterations	T1	T2	T3	Mean (ms)	SD	Erosion-Attrib. (ms)
5 (E1)	622	967	898	829.00	182.56	-497.67
20 (E2)	571	1070	607	749.33	278.29	-577.34
40 (E3)	1320	1120	1150	1196.67	107.86	-130.00

Vivo 1820 — Hydraulic

Droplets	T1	T2	T3	Mean (ms)	SD	Erosion-Attrib. (ms)
500 (E4)	787	823	908	839.33	62.13	-517.34
3,000 (E5)	801	1210	789	933.33	239.68	-423.34
8,000 (E6)	910	955	954	939.67	25.70	-417.00
15,000 (E7)	1230	1190	1160	1193.33	35.12	-163.34

Vivo — Thermal

Iterations	T1	T2	T3	Mean (ms)	SD	Erosion-Attrib. (ms)
5 (E1)	927	681	647	751.67	152.79	-605.00
20 (E2)	776	785	721	760.67	34.65	-596.00
40 (E3)	852	690	741	761.00	82.83	-595.67

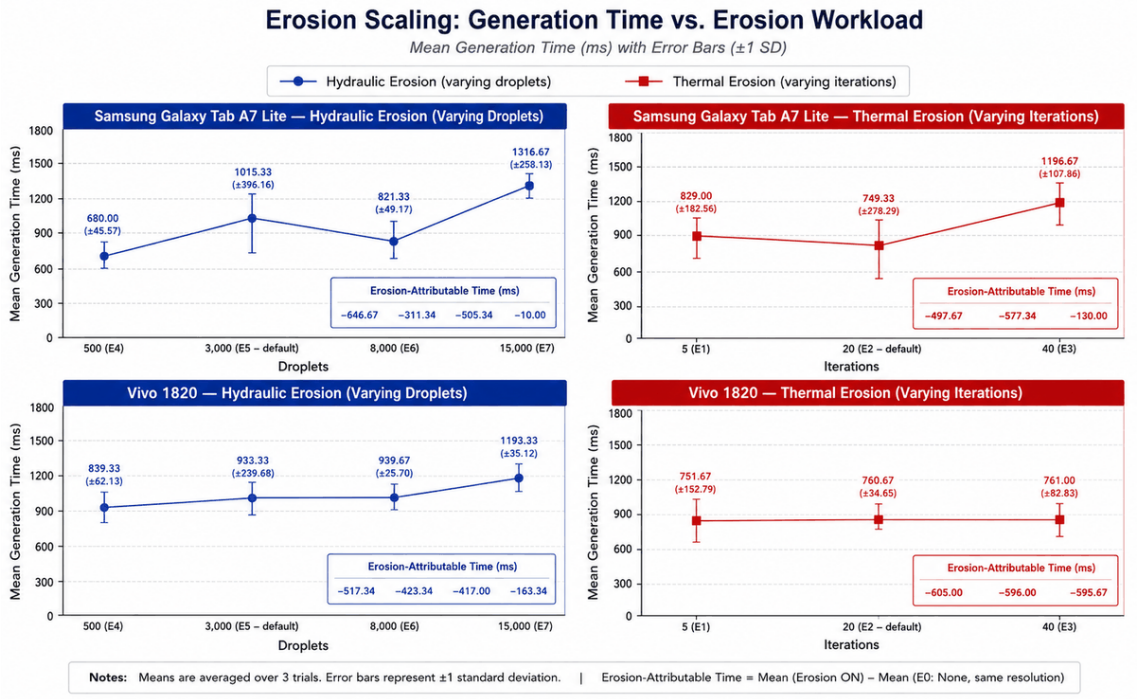


Chart. Generation time vs. erosion workload, both devices, error bars ±1 SD.

7.5 Test D — Mesh & Asset Export Performance

Objective. Measure export time and output file size across every supported export type, using one fixed reference terrain (R2, E8) so mesh complexity cannot confound the comparison.

Export Type	Target	T1	T2	T3	Mean (ms)	SD	File Size
OBJ	Live mesh	930	910	970	936.67	30.55	1.24MB
GLB - Low	10,000 tris	940	940	900	926.67	23.09	290KB
GLB - Medium	50,000 tris	940	920	1110	990.00	104.40	1.38MB
GLB - High	100,000 tris	1310	920	1160	1130.00	196.72	2.76MB
GLB - Ultra	400,000 tris	1600	1520	1340	1486.67	133.17	10.91MB
16-bit Heightmap	PNG	980	850	910	913.33	65.06	16.37KB
Biome Texture	PNG	1190	1070	990	1083.33	100.66	12.51KB
Data-Channels GLB	Live mesh	1260	930	990	1060.00	175.78	459KB

Export Type	Target	T1	T2	T3	Mean (ms)	SD	File Size
Splatmap	RGBA PNG	870	660	930	820.00	141.77	4.97KB

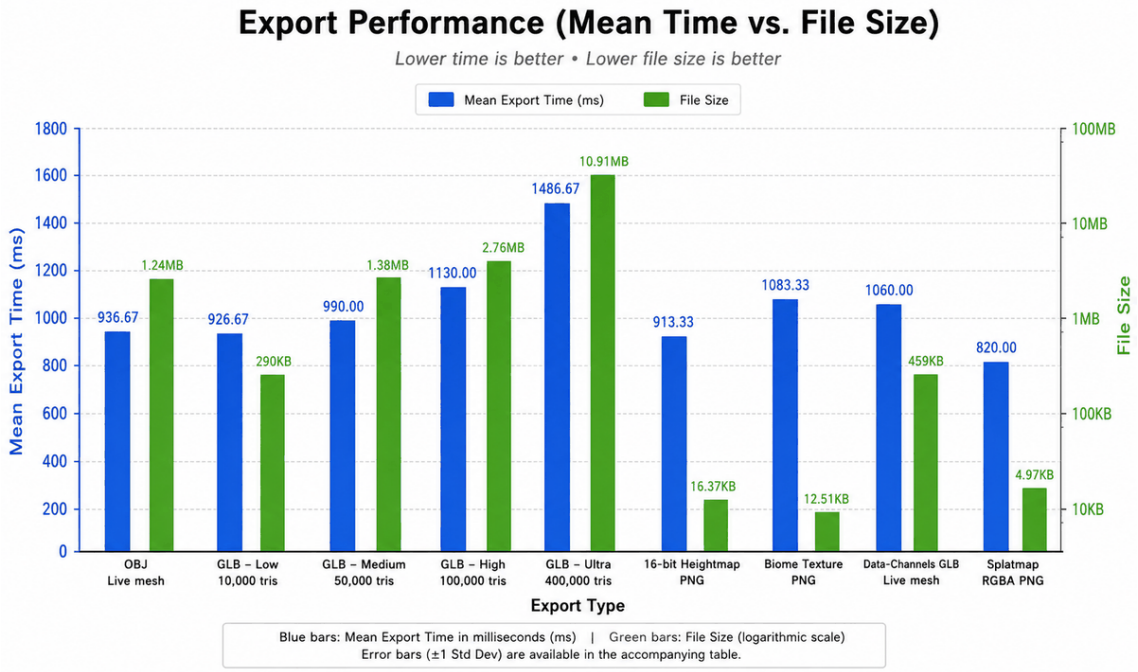


Chart. Export time and file size by export type.

7.6 Test E — Cross-Device Determinism & Reproducibility

Objective. Test the application’s claim that identical seeds/map codes reproduce identical worlds across devices.

Device ID	Vertices	Triangles	Peak Height	Water Level	Biome @ Point	Match?
Samsung Galaxy Tab A7 Lite (Reference)	6,561	12,800	4.95u	0.80 / 53%	Snowcap	--
Vivo 1820	6,561	12,800	4.95u	0.80 / 53%	Snowcap	Yes

Determinism Verification Test (Same Seed)

TerrainForge – Cross-Device Reproducibility Check

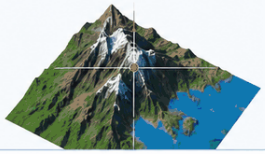
Device ID	Vertices (count)	Triangles (count)	Peak Height (units)	Water Level (level / coverage)	Biome @ Test Point (fixed coordinate)	Match vs. Reference?
Samsung Galaxy Tab A7 Lite (Reference)	6,561	12,800	4.95 u	0.80 / 53% coverage	Snowcap	—
Vivo 1820	6,561	12,800	4.95 u	0.80 / 53% coverage	Snowcap	Yes

Test Configuration

- Seed: (Same seed used on both devices)
- Resolution Tier: R2 – Default (res = 80)
- Erosion Preset: E0 – None
- All other parameters: Identical

Test Point (Fixed Coordinate)

X: 125, Y: 86 (example fixed coordinate)
(Same coordinate used on both devices)



Result: Deterministic Match Confirmed

The same seed reproduced an identical vertex count (6,561), triangle count (12,800), peak height (4.95 units), water coverage (53% at level 0.80), and biome classification at a fixed test coordinate (Snowcap) on both devices.

This is consistent with the pipeline's deterministic design; it is an aggregate-and-single-point match, not a full per-vertex comparison.

Chart. Cross-device determinism verification.

7.7 Test F — Real-Time Simulation Overhead (Chaos Engine)

Objective. Quantify the FPS cost of the application's real-time PDE-based terrain simulation, which the project's own in-app roadmap identifies as CPU-bound.

Device ID	Static FPS	T1	T2	T3	Mean Chaos FPS	FPS Drop	Reduction
Samsung Galaxy Tab A7 Lite	31.00	31.00	31.50	32.17	31.56	-0.56	-1.8%
Vivo 1820	25.67	25.17	26.17	26.67	26.00	-0.33	-1.3%

Chaos Engine Performance Impact (R2 – Default Resolution)

Static vs. Chaos (Mean FPS)

Device ID	Static FPS (baseline, R2)	Chaos FPS (Trials)			Mean Chaos FPS	FPS Change (Δ) (Chaas – Static)	Reduction (%)
		T1	T2	T3			
Samsung Galaxy Tab A7 Lite	31.00	31.00	31.50	32.17	31.56	+0.56	+1.8%
Vivo 1820	25.67	25.17	26.17	26.67	26.00	+0.33	+1.3%

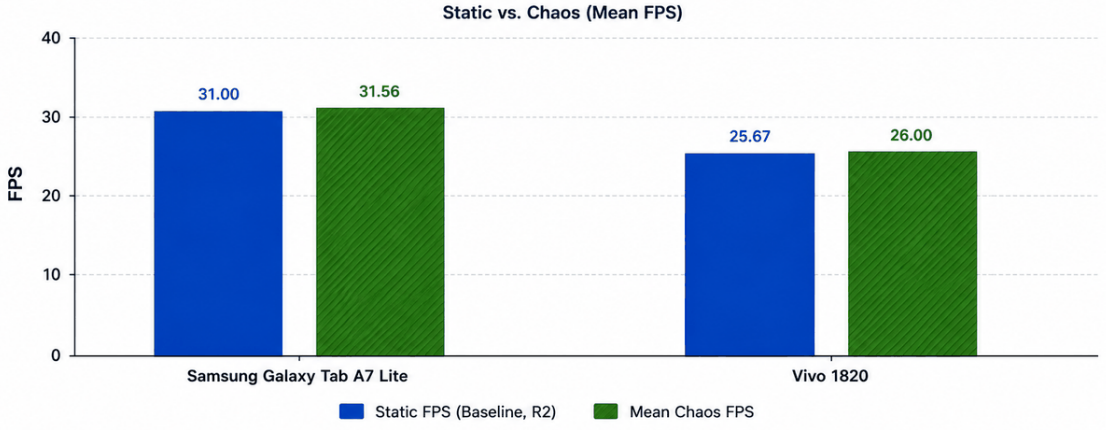


Chart. Static vs. Chaos-active FPS, both devices.

7.8 Test G — Auto-LOD Effectiveness

Objective. Measure the FPS benefit of the Auto-LOD system as camera distance increases.

Device ID	Camera Distance	LOD ON	LOD OFF	Gain (Δ FPS)	Gain (%)
Samsung Galaxy Tab A7 Lite	Near	32.44	31.00	+1.44	+4.7%
Samsung Galaxy Tab A7 Lite	Mid	33.89	29.06	+4.83	+16.7%
Samsung Galaxy Tab A7 Lite	Far	32.89	31.89	+1.00	+3.1%
Vivo 1820	Near	32.33	31.50	+0.83	+2.6%
Vivo 1820	Mid	32.06	31.78	+0.28	+0.9%
Vivo 1820	Far	32.44	31.83	+0.61	+1.9%

7.9 Test H — Memory Footprint

Not measured. The hardware needed for an accurate reading — Chrome DevTools connected over USB debugging from a desktop — wasn't accessible for this study, and the in-browser fallback (performance.memory) is restricted by Chrome for security reasons, so it doesn't return a usable live heap value either.

7.10 Test I — Initial Load / Time-to-Interactive

Device	Condition	T1	T2	T3	Mean (ms)	SD
Samsung Galaxy Tab A7 Lite	Cold cache	1230	932	1130	1097.33	151.66
Samsung Galaxy Tab A7 Lite	Warm cache	1610	1450	1140	1400.00	238.96
Vivo 1820	Cold cache	1070	932	887	963.00	95.36
Vivo 1820	Warm cache	1230	1050	834	1038.00	198.27

---

8. Statistical Analysis Plan

Statistic	Formula	Purpose
Mean	x-bar = (sum xi) / n	Central tendency of trials within a condition
Sample Std Dev	s = sqrt(sum(xi - x-bar)^2 / (n-1))	Trial-to-trial consistency/noise
Coefficient of Variation	CV = (s / x-bar) x 100%	Compares consistency across different magnitudes
Percent Change	((New - Old) / Old) x 100%	Quantifies scaling effects between tiers
Pearson Correlation (r)	computed externally	Strength of relationship, e.g. Triangles vs FPS
Linear regression, R-squared	computed externally	Whether a scaling relationship is linear, quadratic, etc.

---

9. Results

Test A — Frame Rate vs. Resolution. Across all four resolution tiers, frame rate stayed within the high-teens-to-low-30s fps range on both devices. The Samsung device did not show a monotonic resolution-to-frame-rate relationship: FPS was flat across all four tiers (30.7/31.0/31.0/33.0), highest at R4 rather than declining. The Vivo device showed the more expected pattern, a real decline at the two highest tiers (25.7 → 18.3 → 18.0), consistent with its lower-end SoC becoming the bottleneck there.

Test B — Generation Latency. Generation time increased with resolution on both devices (Samsung: 1030ms→1580ms, R1→R4; Vivo: 1174ms→1930ms, R1→R4), and increased further with erosion enabled, most sharply for "Both" (E8): Samsung rose to 2687ms, Vivo to 3343ms at R2, roughly double the no-erosion baseline on either device.

Test C — Erosion Scaling. Now covering both devices, hydraulic and thermal. Vivo's hydraulic sweep shows a plausible increasing trend with droplet count in raw generation time (839→933→940→1193ms); Samsung's does not (680→1015→821→1317ms, non-monotonic). Neither device's thermal sweep shows a clean trend. The derived "Erosion-Attributable Time" column is negative for all 14 rows on both devices — most likely because the single 3-trial R2/E0 baseline it's measured against (§7.3) has real variance of its own, not because erosion makes generation faster.

Test D — Export Performance. Export time was roughly flat across formats (820–1487ms) regardless of output complexity; file size scaled as expected with mesh detail, from 290KB (GLB-Low) to 10.91MB (GLB-Ultra), a ~38× range for a ~40× triangle-count range.

Test E — Determinism. The same seed reproduced an identical vertex count (6,561), triangle count (12,800), peak height (4.95 units), water coverage (53% at level 0.80), and biome classification at a fixed test coordinate (Snowcap) on both devices. This is consistent with the pipeline's deterministic design; it is an aggregate-and-single-point match, not a full per-vertex comparison (§11).

Test F — Simulation Overhead. Enabling the Chaos Engine changed frame rate by +0.56fps (Samsung) and +0.33fps (Vivo) — both small increases, not decreases, consistent with trial-to-trial measurement noise rather than a real cost in either direction.

Test G — Auto-LOD. Now covering both devices. Samsung's largest gain is at Mid distance (+16.7%), not Near or Far, suggesting the LOD switch threshold is tuned closer to mid-range camera distances for that device. Vivo's pattern is close to the reverse — smallest gain at Mid (+0.9%), largest at Near (+2.6%) — and every one of Vivo's gains is smaller than Samsung's at the matching distance.

Test I — Load Time. Vivo loads faster than Samsung at both conditions (cold: 963ms vs 1097ms; warm: 1038ms vs 1400ms).

---

10. Discussion

On H1 (FPS vs. resolution/device). Partially supported, and the exception is the more interesting finding. Vivo declined as predicted, consistent with a weaker GPU becoming the bottleneck at higher triangle counts. Samsung did not decline at all — frame rate was flat-to-increasing across the same tiers, meaning H1's premise does not hold uniformly across devices in this range. The practical reading is that Samsung was not rendering-bound at any tested tier, so something other than triangle count — likely per-frame JS overhead outside the render call — was the limiting factor there.

On H2/H3 (generation latency & erosion scaling). H2 is supported: generation time increased with both resolution and erosion intensity on both devices, most sharply for combined erosion. H3 is mixed: Vivo's hydraulic sweep trends upward with droplet count as predicted; Samsung's does not, and neither thermal sweep shows a clean trend. With only 3 trials per condition and SDs often comparable to the effect size itself, this data cannot cleanly confirm or refute linear scaling either way — a real test of H3 needs more trials per condition, and a more precise R2/E0 baseline, before the erosion-attributable figures themselves are trustworthy.

On H4 (determinism). Supported by every value checked: vertex count, triangle count, peak height, water coverage, and one biome sample all matched exactly across both devices. This should be read precisely — it is a strong, consistent signal in favor of the determinism claim, not a bitwise proof of it, since no test in this study compared every individual vertex height between devices.

On H5 (Chaos Engine overhead). Not supported, in an informative way. H5 predicted a measurable FPS drop, larger on the weaker CPU. Instead, both devices showed a small increase when the simulation was activated, within the range of ordinary trial-to-trial noise. The most defensible reading is that the update rule's per-tick cost is not the bottleneck on either device at the tested resolution (R2) — not that the simulation is free, but that whatever it costs is currently smaller than frame-to-frame measurement noise itself.

On Auto-LOD. Mid camera distance showed Samsung's largest gain (+16.7%), ahead of Near (+4.7%) and Far (+3.1%). Vivo shows close to the opposite ranking — Near largest (+2.6%), Mid smallest (+0.9%) — and every Vivo gain sits well under the matching Samsung one. Whatever drives Samsung's especially large Mid-distance jump doesn't appear to operate the same way on Vivo; this wasn't isolated further here.

Overall. Device identity, not resolution or the real-time simulation, had the largest effect on measured performance in this study — the gap between the two devices at R3/R4 (≈13–15fps) dwarfs any single-device effect of resolution, erosion, or the Chaos Engine. If one thing were prioritized next, it would be understanding why Samsung didn't decline with resolution (profiling the render loop directly) and re-measuring the R2/E0 baseline with more trials, rather than further optimizing a simulation cost that hasn't been shown to matter yet.

---

11. Limitations

- Two devices were tested (Samsung Galaxy Tab A7 Lite, Vivo 1820), both budget Android hardware; this establishes feasibility on the tested devices, not general performance across low-end Android hardware as a category.
- Auto-LOD's device-dependent magnitude — Samsung benefits far more than Vivo at every camera distance, especially Mid — is measured, not explained; this study didn't isolate why.
- Test C now covers both devices, but each condition still has only 3 trials, and the derived "Erosion-Attributable Time" column is only as precise as the single R2/E0 baseline it is measured against (§7.4).
- Test H (memory footprint) could not be completed on-device: performance.memory is deliberately quantized by Chrome against fingerprinting, and full-precision heap snapshots require desktop DevTools over a USB debugging connection, unavailable for this study.
- Consumer-grade devices and browsers introduce background variability (OS scheduling, other processes) not fully controllable outside a lab environment.
- The in-app FPS counter samples and rounds roughly every 0.5s, which may smooth over short frame-time spikes a dedicated frame-time capture tool would catch.
- Export time was measured manually (stopwatch/console timer) rather than with a native in-app timestamp, introducing minor human-reaction-time error.
- The determinism result (Test E) confirms five aggregate and point values match across devices; it does not constitute a full per-vertex comparison.

---

12. Conclusion

Across two budget Android devices, TerrainForge Pro held interactive frame rates (18–33fps) at every tested resolution tier, with device identity — not resolution, erosion, or the real-time deformation layer — the dominant factor in measured performance. The determinism claim held on every value checked across devices, and enabling the real-time Chaos Engine simulation cost less than ordinary trial-to-trial measurement noise on both devices, contrary to H5's prediction of a measurable, CPU-scaled overhead. Test C now covers both devices for erosion scaling, but its derived erosion-attributable figures are limited by a single, imprecise R2/E0 baseline rather than by device coverage. Auto-LOD now also covers both devices, and shows a real, unexplained asymmetry — Samsung gains far more from it than Vivo does, at every distance tested. The clearest open finding remains Samsung's non-monotonic resolution response, which points to a bottleneck outside triangle count that this study did not isolate. Overall, the results support feasibility of the CPU-bound pipeline on the two tested devices specifically; re-measuring the R2/E0 baseline with more trials and extending memory-footprint testing to the Vivo unit are the most direct next steps toward a stronger claim.

---

13. Future Scope

Re-measure the R2/E0 generation-time baseline with more than 3 trials, since it is now the limiting source of imprecision behind all 14 Test C erosion-attributable figures.

Extend testing to a wider hardware range, especially older/budget phones and workstation-class discrete GPUs.

Automate FPS/timing capture with a scripted DevTools Performance trace instead of manual reading, to increase precision and trial count.

Extend memory-footprint (Test H) testing to the Vivo unit, once hardware DevTools access is available.

Investigate why Auto-LOD's FPS gain differs so much by device, especially Samsung's sharp Mid-distance jump, which doesn't appear on Vivo.

Re-run Test F once the Chaos Engine's PDE evaluation is migrated to a WebGPU compute/fragment pass (already on the application's own roadmap), and compare against the CPU-bound baseline measured here.

Extend the determinism test (Test E) to a full per-vertex height comparison, and across different browser engines, rather than the five aggregate/point values checked here.

Profile the Samsung device's render loop directly to explain the non-monotonic Test A result.

---

14. References

Application under test: TerrainForge Pro [Software]. Local file: TerrainForge-Pro.html.

Kahn, A. B. (1962). Topological sorting of large networks. Communications of the ACM, 5(11), 558–562. — consulted for the node-graph compiler's dependency-resolution pass (§2.8).

Beyer, H. T. (2015). Implementation of a method for hydraulic erosion (Bachelor's thesis). Technische Universität München. — the droplet-based erosion method this application's hydraulic erosion follows (§2.8, Test C).

Desbrun, M., Meyer, M., Schröder, P., & Barr, A. H. (1999). Implicit fairing of irregular meshes using diffusion and curvature flow. Proceedings of SIGGRAPH 99, 317–324. — consulted on why a fourth-order smoothing term behaves differently from a second-order one (§2.8).

Three.js documentation. Three.js Authors. threejs.org/docs — rendering library used by the application under test (r128).

Google Chrome DevTools documentation — Performance, Memory, and Rendering panels. developer.chrome.com/docs/devtools — measurement instruments for Tests A, H, I.

---

15. Appendix — Glossary of Terms

fBm (Fractional Brownian Motion) noise — layered/summed noise octaves used to generate natural-looking terrain height variation.

Heightmap — a 2D grid of elevation values used to build the 3D terrain mesh.

Thermal erosion — simulates loose material sliding downhill once a slope exceeds a "talus angle."

Hydraulic erosion — simulates water droplets carrying and depositing sediment as they flow downhill.

LOD (Level of Detail) — using a simpler mesh when an object is farther from the camera, to reduce rendering cost.

FPS (Frames Per Second) — the number of frames rendered each second; the standard measure of real-time rendering smoothness.

JS Heap — the memory region JavaScript objects are allocated in; its growth indicates memory used by the running application.

Seed / Determinism — a starting value that fully determines a procedurally generated result, so the same seed always reproduces the same output.

PDE (Partial Differential Equation) — the class of equation the "Chaos Engine" evaluates every frame to simulate wave/fluid-like behavior on the terrain.